

Resumen de proyecto y código de recibo de sueldo

Estructura relevante del proyecto

Raíz de MiApp (archivos y carpetas relevantes):

- app.json
- package.json
- tsconfig.json
- src/
 - app/
 - (tabs)/
 - recibos.tsx
 - recibo-pdf.tsx
 - components/
 - recibo-row.tsx
 - controllers/
 - use-recibos.ts
 - models/
 - types.ts
 - mock.ts
 - services/
 - recibos-service.ts
 - api-client.ts
 - views/
 - recibos-view.tsx

src/services/recibos-service.ts

```
/**
 * Servicio de recibos de sueldo.
 *
 * ■■■ TEMPLATE: expone la carga y la descarga de recibos. Hoy `listar`
 * devuelve el mock para poblar la UI; reemplazar por el fetch real.
 */

import { recibosMock } from '@models/mock';
import type { Recibo } from '@models/types';

export const recibosService = {
  /** Trae los recibos del empleado autenticado. */
  async listar(): Promise<Recibo[]> {
    // TODO: reemplazar el mock por la llamada real:
    // return apiRequest<Recibo[]>('/recibos');
    return recibosMock;
  },

  /** Descarga (y abre/comparte) el PDF de un recibo. */
  async descargar(recibo: Recibo): Promise<void> {
    // TODO: descargar el archivo desde `recibo.archivoUrl` con
    // expo-file-system y abrirlo/compartirlo (expo-sharing / Linking).
    console.log('TODO: descargar recibo', recibo.id);
  },
};
```

src/controllers/use-recibos.ts

```
/**
 * Controller de recibos de sueldo.
 *
```

```

* ■■■ TEMPLATE: maneja el estado de UI (lista, cargando, error) y delega la
* obtención/descarga en `recibosService`. El fetch real va en el service.
*/

```

```

import { useEffect, useState } from 'react';

import type { Recibo } from '@models/types';
import { recibosService } from '@services';

export interface UseRecibosResult {
  recibos: Recibo[];
  cargando: boolean;
  error: string | null;
  /** Descarga/abre el PDF de un recibo (delega en recibosService). */
  descargar: (recibo: Recibo) => void;
}

export function useRecibos(): UseRecibosResult {
  const [recibos, setRecibos] = useState<Recibo[]>([]);
  const [cargando, setCargando] = useState(true);
  const [error, setError] = useState<string | null>(null);

  useEffect(() => {
    let activo = true;
    setCargando(true);
    recibosService
      .listar()
      .then((data) => {
        if (activo) setRecibos(data);
      })
      .catch(() => {
        if (activo) setError('No se pudieron cargar los recibos.');
```

src/app/(tabs)/recibos.tsx

```

import { useRecibos } from '@controllers/use-recibos';
import { RecibosView } from '@views/recibos-view';

/** Ruta "/recibos". Conecta el controller useRecibos con la view. */
export default function RecibosScreen() {
  const { recibos, cargando, error, descargar } = useRecibos();

  return (
    <RecibosView recibos={recibos} onDescargar={descargar} loading={cargando} error={error} />
  );
}

```

src/views/recibos-view.tsx

```
import Ionicons from '@expo/vector-icons/Ionicons';
import { FlatList, StyleSheet, View } from 'react-native';

import { ReciboRow } from '@components/recibo-row';
import { ScreenContainer } from '@components/screen-container';
import { ThemedText } from '@components/themed-text';
import { Radii, Spacing } from '@constants/theme';
import { useTheme } from '@hooks/use-theme';
import type { Recibo } from '@models/types';

export type RecibosViewProps = {
  recibos: Recibo[];
  onDescargar: (recibo: Recibo) => void;
  loading?: boolean;
  error?: string | null;
};

export function RecibosView({ recibos, onDescargar, loading, error }: RecibosViewProps) {
  const theme = useTheme();

  return (
    <ScreenContainer gap={Spacing.three}>
      <View style={styles.header}>
        <ThemedText type="title">Recibos</ThemedText>
        <ThemedText type="default" themeColor="textSecondary">
          {recibos.length > 0
            ? `${recibos.length} ${recibos.length === 1 ? 'recibo disponible' : 'recibos disponibles'}`
            : 'Tus recibos de sueldo'}
        </ThemedText>
      </View>

      {error ? (
        <ThemedText type="small" themeColor="danger">
          {error}
        </ThemedText>
      ) : null}

      <FlatList
        data={recibos}
        keyExtractor={({ item }) => item.id}
        renderItem={({ item }) => <ReciboRow recibo={item} onDescargar={onDescargar} />}
        contentContainerStyle={styles.list}
        showsVerticalScrollIndicator={false}
        ListEmptyComponent={
          <View style={styles.empty}>
            <View style={[styles.emptyIcon, { backgroundColor: theme.backgroundElement }]}>
              <Ionicons name="receipt-outline" size={40} color={theme.textSecondary} />
            </View>
            <ThemedText type="default" themeColor="textSecondary" style={styles.emptyText}>
              {loading ? 'Cargando...' : 'No hay recibos disponibles.'}
            </ThemedText>
          </View>
        }
      />
    </ScreenContainer>
  );
}

const styles = StyleSheet.create({
  header: {
    gap: Spacing.one,
  },
  list: {
    gap: Spacing.three,
    paddingTop: Spacing.two,
  },
  empty: {
    alignItems: 'center',
  },
});
```

```

    gap: Spacing.three,
    paddingTop: Spacing.six,
  },
  emptyIcon: {
    width: 88,
    height: 88,
    borderRadius: Radii.pill,
    alignItems: 'center',
    justifyContent: 'center',
  },
  emptyText: {
    textAlign: 'center',
  },
},
});

```

src/components/recibo-row.tsx

```

import { StyleSheet, View } from 'react-native';

import { Card } from '@components/card';
import { IconBadge } from '@components/icon-badge';
import { IconButton } from '@components/icon-button';
import { ThemedText } from '@components/themed-text';
import { Spacing } from '@constants/theme';
import type { Recibo } from '@models/types';

type ReciboRowProps = {
  recibo: Recibo;
  onDescargar: (recibo: Recibo) => void;
};

export function ReciboRow({ recibo, onDescargar }: ReciboRowProps) {
  return (
    <Card style={styles.card}>
      <IconBadge name="document-text-outline" size={48} />

      <View style={styles.info}>
        <ThemedText type="default" style={styles.periodo} numberOfLines={1}>
          {recibo.mesPago}
        </ThemedText>
        <ThemedText type="small" themeColor="textSecondary">
          Recibo de sueldo
        </ThemedText>
      </View>

      <IconButton
        accessibilityLabel={`Descargar recibo de ${recibo.mesPago}`}
        onPress={() => onDescargar(recibo)}
      />
    </Card>
  );
}

const styles = StyleSheet.create({
  card: {
    flexDirection: 'row',
    alignItems: 'center',
    gap: Spacing.three,
    padding: Spacing.three,
  },
  info: {
    flex: 1,
    gap: Spacing.half,
  },
  periodo: {
    fontWeight: '700',
    fontSize: 17,
  },
},

```

```
});
```

src/app/recibo-pdf.tsx

```
import { useLocalSearchParams } from 'expo-router';
import { useEffect, useState } from 'react';
import { ActivityIndicator, View } from 'react-native';
import { WebView } from 'react-native-webview';

import { API_BASE_URL, authService } from '@services';

export default function ReciboPdfScreen() {
  const { id } = useLocalSearchParams<{ id: string }>();
  const [token, setToken] = useState<string | null>(null);

  useEffect(() => {
    (async () => {
      const t = await authService.obtenerToken();
      setToken(t);
    })();
  }, []);

  if (!id || !token) {
    return (
      <View style={{ flex: 1, justifyContent: 'center', alignItems: 'center' }}>
        <ActivityIndicator />
      </View>
    );
  }

  const uri = `${API_BASE_URL}/api/v1/reciboDeSueldo/pdf/${id}`;

  return (
    <View style={{ flex: 1 }}>
      <WebView
        source={{
          uri,
          headers: {
            Authorization: `Bearer ${token}`,
          },
        }}
      />
    </View>
  );
}
```